



LLM世界引擎：打造高自由度 AI 虛擬世界的架構與實作歷程

這是什麼&&我學到了什麼

這個項目的目標是打造一個可以讓ai角色自由行動的世界。有多自由？我唯一給世界系統的約束就是要在平面座標上，其他不管ai角色想做什麼事都可以，比如：找其他ai角色聊天、出門、做早餐、打架或是看電視。這套系統的最終目標是成為遊戲的核心，所以有座標與狀態等資訊方便連接遊戲引擎。

在這段開發歷程中，我學到了：

- **程式架構設計與除錯**：學會用紙筆先把所有細節寫出來再放到腦袋裡模擬，也體會到過度複雜的技術（如 Asyncio）對維護的副作用與技術債。
- **物件導向設計**：將人物、環境甚至世界主體全部抽象化為物件，獨立各自的屬性與方法。
- **提示詞工程技術**：如何引導語言模型處理複雜指令。

系統AI與架構重構的契機

原本我的世界只有角色ai，然後讓他們可以講話互動或是做自己的事，但後來我發現這樣做角色只能「做」而沒有「回饋」，且大型語言模型很容易出現幻覺，這導致整個概念都變得很失敗。於是我直接把程式全部重寫，並發明世界ai（系統ai），來輔助角色ai。

當角色ai想對環境做任何事的時候，他都會把自己的動作發送給世界ai。用「開門」來舉例：

- 當角色要開門時，他會告訴世界ai：我要開門。
- 世界ai收到訊息後，會更新門的狀態，讓門打開。然後做出一個訊息：你成功把門打開了，發送回給角色ai。

與其他ai角色互動是另外一種模式：

- 當角色A對角色B打招呼想說你好，角色A會告訴系統：我對B說你好。
- 這時候系統會結合狀態跟動作修飾訊息然後告訴角色B：角色A揮著手對你說你好。

這樣做使得ai角色收到反饋後比較不會出現幻覺，且角色幾乎想做什麼就可以做什麼。用系統ai

來控制狀態穩定性及可行性，會比讓角色自己控制好很多。

核心系統實作

描述與位置系統

因為llm是語言模型，只能看到文字，所以對物件、人物、環境的理解都要依賴描述。我設計了「強制描述」（如名字、性別、相對位置）與「開放式描述」（如穿搭、表情）。其中相對位置比較有趣，原本我是直接提供ai絕對位置比如 (3,4) 但效果不佳，後來我寫了一個位置系統，把絕對位置變成「距離/角度」的方式給ai使用，加上他站在什麼物件旁邊的描述來當作輔助。

環境系統與分區加載

環境系統就像很多個小盒子，而物件跟人物只能放在一個小盒子裡面。除了用來作為角色與物體互動的中介，還有一個技術上的原因是：假如我的整個世界太大，角色就會看到太多東西從而讓速度變慢跟ai花費提升，有了這套系統就可以有分區加載的效果。

記憶管理與暫存事件

當ai在做任何事時，他會持續把自己生成的記憶存入「暫存記憶」，並在事情做完後整理存入「短期記憶」，等一天結束睡覺時再整理到「長期記憶」，這讓ai可以完整地做完一件事且不容易出現幻覺。另外，當A角色閒置（暫停）的時候，B角色做了某些事情的環境變化會被當作「暫存事件」，等下次A角色運作時就能接收到剛剛發生的事。

展示

```
○ (.venv) kelsier@eva01:~/py/ai_text_game$ /home/kelsier/py/ai_text_game/.venv/bin/python /home/kelsier/py/ai_text_game/async.py
Alice turned to Emily and said: 'Good morning, Emily!'
Emily smiled at Alice and said: 'Good morning, Alice! Did you sleep well?'
Alice said to Emily: 'Yes, I slept really well. How about you?'
Emily said to Alice: 'I slept well too, thanks for asking. Do you have any plans for today?'
Alice said to Emily: 'I was thinking of practicing basketball for a bit and then finishing my math homework. What about you?'
Emily said to Alice: 'That sounds like a good plan! I was thinking of visiting my favorite café and then working on my proposal for the meeting on Thursday.'
Alice said to Emily: 'That sounds like a great plan! Maybe we can meet up later after we're done with our tasks?'
Emily said to Alice: 'That sounds perfect! Let's meet up later then.'
Alice said to Emily: 'Great! I'll see you later then. Have a wonderful time at the café!'
Emily walks to the table, sits down, and starts studying.
Alice starts walking toward the door.
Alice walks toward the door and opens it, entering the living room.
AI: ... to make breakfast.
-- Open file in editor (ctrl + click)
> /home/kelsier/py/ai_text_game/async.py(377):run_env()->None
-> breakpoint()
(Pdb) █
```

這邊只展示了一點點，但可以看到有

- 人物互動 他們講話
- 物件互動 有人跑去讀書
- 環境切換 有人離開房間跑去廚房做飯

角色ai跟系統ai的互動因為太大量了所以隱藏在後台，但他們其實一直在互動，系統ai會不斷更新環境狀態跟角色狀態，然後給角色ai反饋，角色ai則會根據反饋來決定下一步要做什么事。

目前的問題與失敗原因

- **幻覺**：如同我前面一直提到的，ai角色很容易出現幻覺問題，就是他明明沒有做某件事，卻自行把他加入記憶中，我已經盡量避免了，但還是會發生。
- **覆讀機**：最大的問題就是提示詞問題，當ai角色進行10幾次感知後他會開始無視最新事件而一直重複說同一句話。目前推測是因為記憶太多而導致的失焦，這是一個到最後我都沒有解決的問題。
- **非同步 (asyncio)**：這是我認為導致專案失敗的另一個很大的原因。因為調用ai很花時間，為了加速，我一開始就為了這單一功能直接導入了 asyncio 來協程發送請求。但這讓整個本就不單純的程式碼架構變得太過複雜，就算我已經全部重寫三四次了，過了半年後的現在完全不想出手修改，我寧願直接再重寫一次。這也讓我學到了不該在架構不穩定的初期貿然引入過度複雜的技術。

心得與反思

這個項目是我第一次純粹因為覺得很有趣而大力開發的，其實我也挺享受設計架構與實作過程的。雖然一開始因規劃得不夠詳細，導致我為了新功能只能反覆重構程式，但這也**讓我學會了要做這種大型項目一定要先把想法理清了再開工。**

在技術面上，除了asyncio，我還**學到了很多寫ai提示詞的技巧**。因為我的指令相當複雜，要如何讓ai穩定處理成了重點。為此我上網研究，並實際利用在 ais3 學到的ai提示詞工程(prompt engineering)，這讓我具備了引導語言模型穩定處理複雜問題的能力，對我日後開發ai相關程式幫助極大。此外，我也**學會了如何獨自構思複雜的系統架構**。在經歷第三次重寫時，**我已經學會先拿出紙和筆把所有細節寫出來，再放到腦袋裡沙盤推演模擬，以免有邏輯上的錯誤。這個過程雖然相當燒腦，但我也覺得推敲設計架構的過程十分有趣。**

還有一個很深的感悟是，還記得當初剛完工時，我覺得自己的程式架構已經相當漂亮、完美了，但我現在回頭寫筆記，卻能一眼看出問題一堆，甚至不願意修改而是想直接全部再重寫一次。這

種前後心態的反差，讓我發現在技術眼界與程式思維上，的確有在慢慢地進步。

雖然系統還不能算是完整運行，但我覺得在我放棄之前，這都不能叫作一個失敗品。我仍然覺得我的思路跟解決方法是相當有前途的，我也沒有忘記要讓它接入遊戲的最終目標。總結來說，這是一個讓我收穫滿滿且非常有趣的項目，我很喜歡。